

Openclaw 最细部署指南，30 分钟手把手教程



阅读导航：

你是谁	从哪开始看
Mac 用户	第二章准备篇 → 第三章 Mac 篇 → 第五章通用配置
Windows 用户	第二章准备篇 → 第四章 Windows 篇 → 第五章通用配置
已装好想进阶	直接跳到第六章进阶配置

不管你是哪类读者，都先看完第二章准备篇。里面的四个概念科普会让你后面每一步都看得更明白。

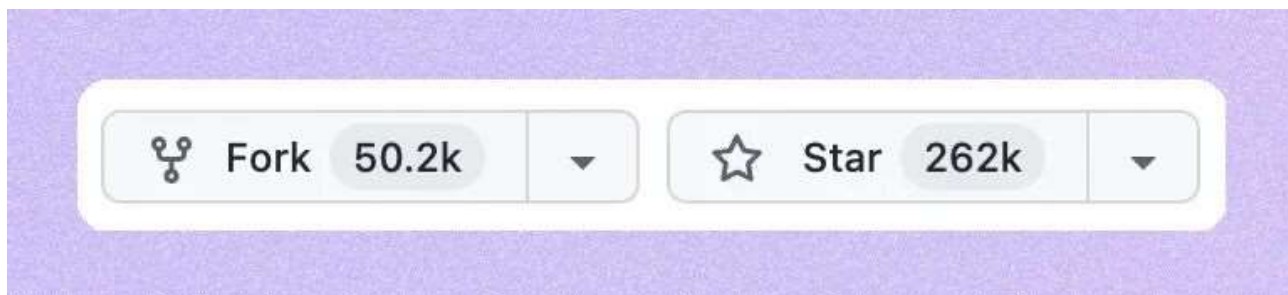
话不多说，开始。

一、OpenClaw 到底是什么？先搞清楚你在装什么

在正式动手之前，我想先花两分钟，跟你说清楚你要装的这个东西到底是啥。

因为我发现，很多人是跟风来装的，"听说很厉害""群里都在聊"，但具体 OpenClaw 是干什么的，其实没太搞明白。

搞明白你在装什么，比装它本身更重要。



先看一组数据。

OpenClaw 在 GitHub 上的 Star 数是 **262,000+**。

Fork 数 50,200+。

MIT 协议开源，意思是完全免费，代码公开，谁都能用，谁都能改。

这是什么概念呢？它是 GitHub 历史上增长最快的开源项目之一，上线不到一个月，就突破了 19 万 Star。这个速度，在整个开源社区的历史上都极其罕见。关键是他已经破圈了，是全民生态下的**最热门项目，不对，不是项目，是切切实实对工作有价值的工具**。

但 Star 数说了不算。关键是，**它到底能帮你干什么？**

你用过 ChatGPT、Claude、DeepSeek 对吧？你让它帮你整理桌面文件，它给你列个一二三四五的步骤——但不会真的动手。

它会说，但它不会做。

OpenClaw 不一样。

OpenClaw 不是聊天机器人。它是一个真正能帮你干活的数字管家。

它做的事情是，**给 AI 装了一部手机。**

让 AI 能通过飞书、Telegram、Discord、WhatsApp 等 20 多个聊天平台，随时随地帮你干活。

不只是聊天。是真的动手干。

你可以在飞书上跟它说"帮我把下载文件夹里的 PDF 按日期归档"，它就真的扫描你的下载文件夹，按日期创建子文件夹，把 PDF 一个个移过去。

你可以在 Telegram 上跟它说"读一下桌面上那篇论文，给我写个 500 字的中文摘要"，它就真的读了论文、理解了内容、写好了摘要、存成了文件。

你可以在微信上跟它说"每天早上 8 点帮我查一下今日 AI 新闻，整理成摘要发给我"，它就真的每天准时推送。

说白了，别的 AI 只会动嘴，OpenClaw 真的动手。

"那它和 Claude Code 有什么区别？"——Claude Code 是给程序员写代码用的闭源工具，OpenClaw 是给所有人处理日常事务用的开源框架。**定位完全不同。**

这个"Open"，**意思是开源，对所有人开放。**记住这个词，后面收尾的时候我还要聊它。

对了，OpenClaw 的创始人 Peter Steinberger 今年 2 月被 OpenAI 挖走了。但社区依然活跃——GitHub 上每天都有新提交，2 月份单月新增 Issue 超过 800 条。**创始人被抢走了，项目反而更火了。**开源的韧性，就在这里。

好了，现在你知道你要装什么了。

一个 **247,000+Star**、支持 20 多个聊天平台、本地部署数据不出设备、完全免费开源的 AI 数字管家。

说实话，光这几个标签，就够让人兴奋了。

二、准备篇——你需要知道的 4 个基础概念

别急着动手。

我知道你现在恨不得马上开始装。但请给我 5 分钟，让我把 4 个基础概念跟你说清楚。

这 5 分钟的投入，能让你后面节省至少 1 个小时的困惑和焦虑。

这玩意就像游戏教程，你跳过教程确实能省两分钟，但后面打 Boss 的时候你会被虐得怀疑人生。

2.1 什么是终端/命令行？

这是我被问得最多的问题。"终端是什么？""命令行是什么？""那个黑底白字的东西是什么？""是不是电影里黑客用的那个？"

别怕。真的别怕。

终端就像你给电脑发微信，你打字发指令，电脑收到后执行。它不说"好的亲"，直接把结果吐给你。

就这么简单。

你平时操作电脑，是用鼠标点来点去对吧？点文件夹、双击应用、拖拽文件。

终端做的是完全一样的事，只不过你不用鼠标了，用打字代替了点击。

比如你想看看桌面上有什么文件：

- 用鼠标？打开 Finder（Mac）或者资源管理器（Windows），点进桌面文件夹，一目了然。
- 用终端？输入一行命令 `ls ~/Desktop`（Mac）或 `dir Desktop`（Windows），文件列表就打出来了。

效果一模一样。只是操作方式不同。

那为什么不用鼠标点呢？为什么要用命令行？

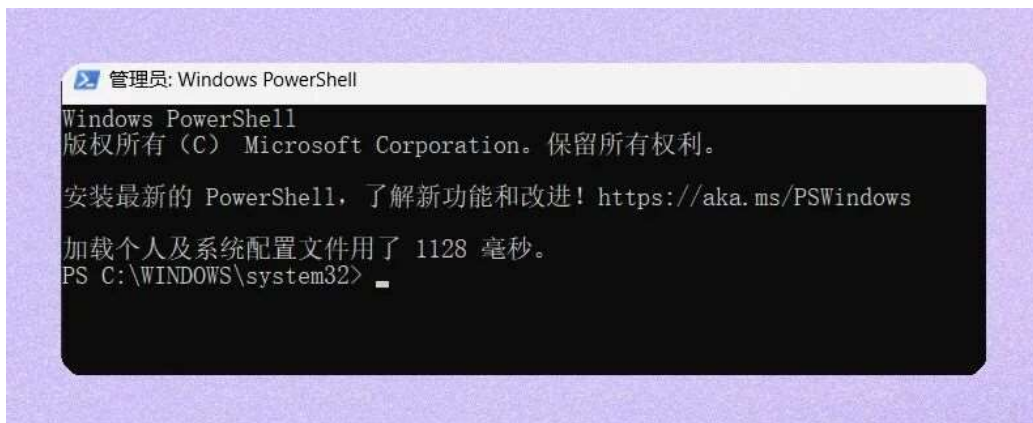
因为有些操作，比如安装软件包、配置环境变量、启动服务，用鼠标点不了，必须用命令行。就像有些表格操作必须用公式而不能手动填一样。



在 Mac 上，这个东西叫"终端"（Terminal）。

怎么打开？最快的方式：按 **Command + 空格**（会弹出 Spotlight 搜索），然后输入"终端"两个字，回车就打开了。

或者你也可以在"应用程序 → 实用工具"文件夹里找到它。



在 Windows 上，它叫"PowerShell"。

怎么打开？按 **Win + R**（会弹出"运行"对话框），输入 **powershell**，回车。

或者更简单：直接在 Windows 左下角的搜索栏里搜"PowerShell"，点击打开。

看到那个窗口了吗？

不管是黑底白字还是白底黑字还是蓝底白字，它就是一个打字的窗口而已。**你接下来要做的事情，90%都是复制粘贴，把我给你的命令复制进去，回车，等它跑完。**

2.2 什么是 Node.js？为什么需要它？

安装 OpenClaw 的第一步，是安装一个叫 Node.js 的东西。之前的文章有介绍过，为了保持完整性，这里再强调一遍～

"这又是什么？为什么要装这个？OpenClaw 不是主角吗？"

好问题。

Node.js 就像一个翻译官。

OpenClaw 是用 JavaScript 这门编程语言写的。但你的电脑原生不认识 JavaScript，就像一个只会中文的人，面前摆了一本法语小说，每个字母都认识但连成句子就完全看不懂。

Node.js 就是这个翻译官，它把 JavaScript"翻译"成你的电脑听得懂的机器语言，让 OpenClaw 能在你的电脑上跑起来。

没有 Node.js，OpenClaw 就是一堆电脑看不懂的天书。

有了 Node.js，OpenClaw 才能正常工作。

所以 Node.js 是"前置依赖"，先装它，才能装 OpenClaw。

版本要求：Node.js ≥ 22 。

这个就像微信要求你的手机系统是 iOS 15 以上、Android 10 以上一样。OpenClaw 要求你的 Node.js 版本不能低于 22。低了它启动不了。

还有一个你会经常看到的名词，**npm**。

npm 全称是 Node Package Manager，Node 的"包管理器"。

你可以把 npm 理解成 Node.js 自带的"应用商店"。

你安装 Node.js 的时候，npm 会自动一起装好。你不需要单独安装它。

等会儿我们安装 OpenClaw 的时候，就是通过 npm 来装的，就像你在 App Store 里搜索一个 App 然后点"获取"一样，只不过换成了在终端里输入一行命令。

Node.js 是翻译官，npm 是应用商店。记住这两个比喻，后面的步骤你就不会懵了。

2.3 什么是 API Key？为什么需要它？

这个概念非常重要。如果你不理解 API Key，后面配置的时候你一定会一脸懵。所以请认真看完这段。

先说结论：**OpenClaw 本身是免费的。完全免费。你不用花一分钱。**

但 OpenClaw 本身并不"聪明"。

它是一个框架，一个"骨架"。它能帮你接收消息、执行任务、管理插件，但它没有"大脑"。

它的"大脑"是外接的，来自 AI 大模型（比如通义千问、Claude、GPT、Kimi 等）。

你对 OpenClaw 说"帮我写一首诗"，OpenClaw 自己写不了。它需要把你的请求转发给 AI 大模型，大模型思考完了把结果返回给 OpenClaw，OpenClaw 再把结果告诉你。

而要调用这些大模型的服务，你需要一个身份凭证。

这个身份凭证，就叫 API Key。

API Key 就像健身房的会员卡，器材（大模型）不是你的，但你刷卡就能用。

具体的工作流程是这样的：

1. 你对 OpenClaw 说："帮我写一首关于春天的诗"
2. OpenClaw 收到请求，拿着你的 API Key 去找 AI 大模型（比如 MiniMax）
3. MiniMax 的服务器验证这个 Key，"验证通过，是合法用户，放行"
4. MiniMax 开始思考，生成了一首诗
5. 把诗返回给 OpenClaw
6. OpenClaw 把诗发送给你

整个过程中，你的对话内容、文件数据都在你自己的设备上。API Key 只是用来向大模型"证明你是谁"的，不会泄露你的数据。

"那 API Key 怎么获取？要花钱吗？"

好消息来了。

这里有个容易踩的坑：OpenClaw 是 AI Agent，不是普通聊天工具。 它每次执行任务，可能要来回调用模型好几轮，Token 消耗比你手动打字聊天大得多。

MiniMax 推出的 Coding Plan，是专门针对这类高频调用场景设计的开发者套餐，**单价低、用量足，日常运行 OpenClaw 基本够用**，不会动不动就超额报错。
(**关键的来了，新用户认证后赠送 15 元的体验金**)



后面配置向导那一步，我会手把手教你怎么获取 API Key（以 MiniMax Coding Plan 为例）。截图、步骤、注意事项全部拉满。

现在你只需要知道，选对套餐，才不会跑着跑着就"断粮"。先把 Coding Plan 办上就行。

其他模型（Claude、GPT 等）的 API Key 按 Token 计费，Agent 跑多了费用上去很快。等你用 MiniMax 玩熟了、摸清自己用量了，再考虑升级也不迟。

2.4 整体流程预览——就 5 步，一目了然

好，四个概念讲完了。

终端 = 给电脑发微信。Node.js = 翻译官。npm = 应用商店。API Key = 会员卡。

接下来，我们的整个安装过程，总共就 5 步：

步骤	干什么	预估时间	难度
第 1 步	安装 Node.js（装翻译官）	5-10 分钟	简单
第 2 步	安装 OpenClaw（装主程序）	3-5 分钟	简单
第 3 步	运行配置向导（设置偏好）	5-10 分钟	中等
第 4 步	获取 API Key + 配置模型	10-15 分钟	中等
第 5 步	首次运行、验证成功	2-3 分钟	简单

Mac 用户还需要在第 1 步之前装一个 Homebrew（Mac 的命令行应用商店），多加 5-10 分钟。

Windows 用户需要额外装一个 Git（代码工具），多加 3-5 分钟。

总计：大约 30-45 分钟。

对，就 5 步。

你想想，你第一次注册微信的时候，也是"下载 App → 输手机号 → 填验证码 → 设置头像 → 开始用"。是不是也是 5 步？

OpenClaw 的安装，本质上就是同一个事。

2.5 两个提前说清的小知识

在正式开始安装之前，还有两件事我先说清楚，后面 Mac 篇和 Windows 篇都会用到。

第一件：npm 镜像源设置（国内用户必做）

npm 默认从国外服务器下载东西，在国内访问可能很慢甚至超时。解决方案是把下载源换成国内镜像（npmmirror.com，原淘宝镜像），速度快十倍以上。就像你下载文件选了个国内 CDN 节点，不走国际线路了。后面安装 OpenClaw 之前，我会提醒你执行这一步。

第二件：WARN 和 ERROR 的区别

安装过程中你会看到终端刷出各种信息。记住两个颜色：

- 黄色的 **WARN**: 友善提醒, "嘿, 有个小事提醒你一下, 但不影响安装。"—不用管。
- 红色的 **ERROR/ERR**: 出事了, 需要处理。——看到了再往下翻常见问题排查。
(实在不行, 把截图发给 ai, 让他指引你一步步完成)

只要没看到红色 ERROR, 就安心等它跑完。

好了, 该动真格了。

Mac 用户请继续往下看第三章 Mac 部署篇。

Windows 用户请直接跳到第四章 Windows 部署篇。

三、Mac 部署篇——从零开始手把手教你

Mac 用户看这里。

如果你是 Windows 用户, 请跳过这整章, 直接去第四章。这一章的每一个命令、每一张截图都是 Mac 专属的, Windows 上操作方式不一样。

这一章我会带你走完从打开终端到成功安装 OpenClaw 的完整过程。

准备好了吗?

打开你的终端, 按 **Command + 空格**, 输入"终端", 回车。

看到那个窗口了吗?

光标在闪？

好。我们开始。

3.1 第一步：安装 Homebrew (Mac 专属)

Homebrew 是什么？

前面讲了，npm 是 Node.js 的应用商店。Homebrew 呢？

Homebrew 是 Mac 整个系统的"命令行应用商店"。

你在 App Store 用鼠标点击就能装 App 对吧？Homebrew 也是装东西的，只不过换成了在终端里输入一行命令。更快、更灵活。

后面我们要通过 Homebrew 来安装 Node.js。所以 Homebrew 是第一步。

如果你已经装过 Homebrew 了，打开终端输入 `brew --version`，如果能看到版本号（比如 `Homebrew 4.x.x`），说明你已经有了，直接跳到 3.2。

如果没装过，或者输入后提示"command not found"，跟着我来。

在终端里，复制粘贴下面这行命令（整行全部复制，一个字符都别漏），然后按回车：

```
curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh
```

这行不用管原理，复制粘贴回车就完了。它做的事情就是从 Homebrew 官方服务器下载安装脚本，安全可靠。

回车之后，终端会提示你输入 Mac 的开机密码（就是你每次开机/安装软件时输入的那个密码）。

注意，这里有一个让无数新手崩溃的"陷阱"：

你打密码的时候，屏幕上不会显示任何字符。没有星号、没有小圆点、什么都没有。

这是正常的。Mac 的终端就是这样设计的，出于安全考虑，不显示密码的任何痕迹。

所以不要以为键盘坏了或者没输进去。你只管正常打完密码，然后按回车就行。

"我打了密码但什么都看不见，是不是输错了？"大概率没输错。直接回车试试。如果密码确实错了，它会再让你输一次。

输完密码后，Homebrew 开始下载和安装。

这个过程大概需要 3-10 分钟，取决于你的网速。

说实话，我自己第一次装 OpenClaw 的时候，就是在这一步翻了车。Homebrew 装了 40 分钟，我以为电脑死机了差点强制关机。后来才知道，是因为默认源在 GitHub，国内访问慢得要命。

Apple 的命令行工具安装速度，慢得像用信鸽传输。

所以如果你也卡住了，别慌，接着往下看。

你会看到终端里哗哗地刷出一堆文字。不用细看，也不用看懂。让它跑就行。

只要没有出现红色的 **ERROR** 字样，就没问题。

当你看到 **Installation successful!** 或类似的成功提示时，恭喜你，Homebrew 装好了。

验证一下：在终端里输入：

```
brew --version
```

如果看到类似 **Homebrew 5.x.x** 的版本号，确认完毕，第一个任务搞定。

进度：1/5，环境准备的第一关过了。

你看，命令行不咬人。它只是长得凶。

常见问题：Homebrew 安装过程中一直卡住不动？

如果安装过程中一直卡在 "Downloading..." 或者 "Fetching..." 不动，大概率是网络问题。因为 Homebrew 的默认源在 GitHub（国外服务器），国内访问有时候会很慢甚至超时。

解决方案：换成国内镜像源安装。在终端输入以下命令：


```
/bin/zsh -c "$(curl-fsSLhttps://gitee.com/cunkai/HomebrewCN/raw/master/Homebrew.sh)"
```

这是一个国内开发者维护的 Homebrew 安装脚本，走的是 Gitee（国内代码托管平台）的镜像，下载速度会快很多。

执行后它会让你选择镜像源，选哪个都行，推荐选"中科大"或"清华"。

好，Homebrew 搞定了。

3.2 第二步：安装 Node.js

有了 Homebrew，安装 Node.js 就是一行命令的事。

在终端输入：

```
brew install node
```

一句话：告诉 Homebrew 帮我装 Node.js。

Homebrew 会自动帮你完成以下所有步骤：

1. 从镜像服务器下载 Node.js 安装包
2. 解压并安装到正确的目录
3. 配置好环境变量（让你在终端里能直接用 `node` 和 `npm` 命令）

你什么都不用管。就等着它跑完就行。

安装过程大概 1-3 分钟。

跑完之后，验证一下：

```
node -v
```

如果看到 **v22.x.x** 或更高的版本号，完美。Node.js 安装成功。

再验证一下 npm（应用商店）：

```
npm -v
```

看到版本号就行。npm 是 Node.js 自带的，装了 Node.js 就自动有了，不需要单独安装。

进度：2/5，翻译官到位了。

两行命令，一个翻译官就位。这节奏比点外卖还快。

如果你的 Node.js 版本低于 22 怎么办？

比如你之前装过旧版本的 Node.js，`node -v` 显示的是 **v18.x.x** 或 **v20.x.x**。

解决方案：

```
brew upgrade node
```

这行命令会把 Node.js 升级到 Homebrew 能提供的最新版本。

升级完再 `node -v` 验证一下，确认版本号 `>= 22`。

如果升级后版本还是不对（极少见情况），试试：

```
brew unlink node && brew link node --overwrite
```

两行命令，搞定。继续。

3.3 第三步：安装 OpenClaw

好，重头戏来了。主角登场。

安装 OpenClaw 有两种方式：

方式一：官方一键脚本（推荐）

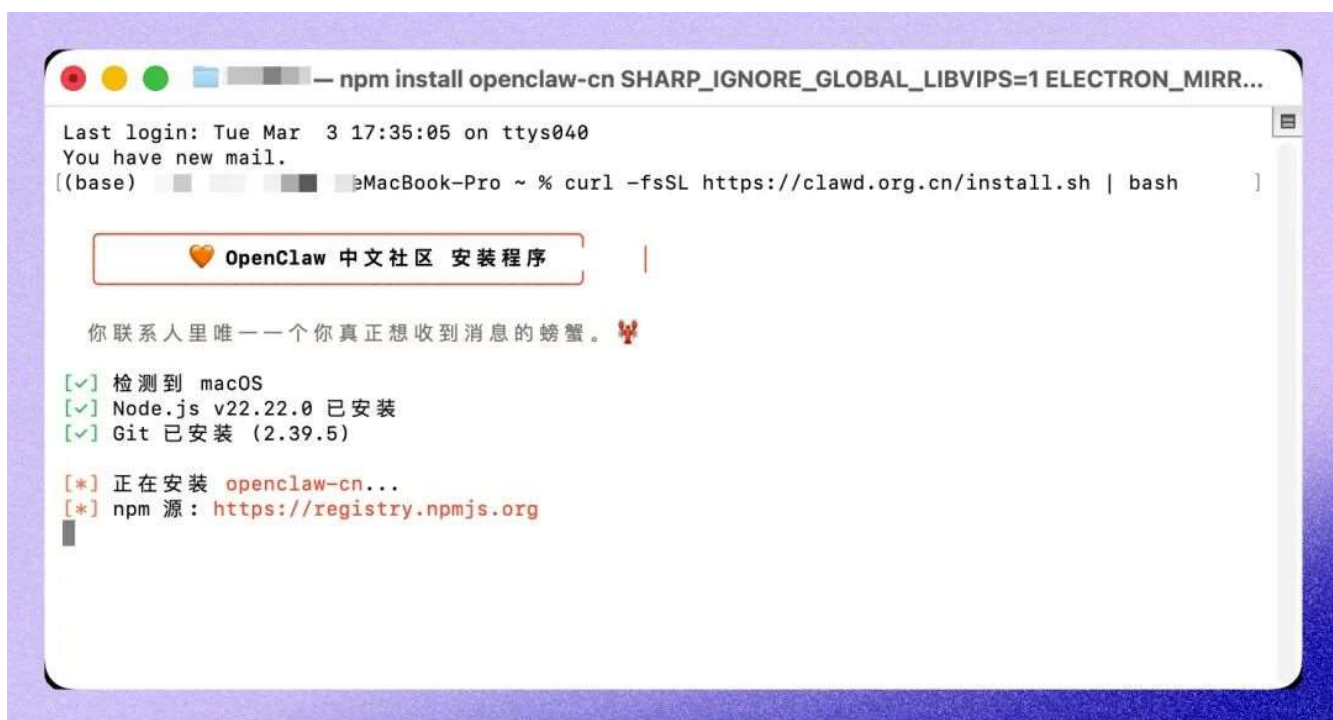
```
curl -fsSL https://clawd.org.cn/install.sh | bash
```

方式二：通过 npm 安装（备选）

```
npm install -g openclaw-cn@latest
```

我推荐方式一。这是官方中文社区维护的安装脚本，一行命令搞定所有依赖配置，路径、权限、环境变量全部自动处理好，出了问题也容易定位。

方式二的 npm 安装适合已经熟悉 Node.js 生态的用户。如果你是第一次接触，直接用方式一。



回车后脚本会自动运行，你会看到终端里刷出安装进度。

老规矩：黄色 WARN 不用管，只有红色 ERROR 才需要处理（准备篇讲过的）。

安装完成后（通常 1-3 分钟），验证一下：

```
openclaw-cn --version
```

如果看到版本号，比如 `openclaw-cn/2026.2.23` 或类似的，你已经成功在 Mac 上安装了 OpenClaw 主程序。

进度：3/5，主角登场了。最难的部分已经过了。

一行命令、验版本，两步。说实话，很多人想象中的"安装部署"是一座大山，走到这才发现——也就两级台阶。

常见问题：报 EACCES 权限错误？

报错长这样：`Error: EACCES: permission denied`

意思是：你没有权限往这个目录里装东西。

解决方案：在命令前面加 `sudo`（管理员权限运行）：

```
sudo curl -fsSL https://clawd.org.cn/install.sh | bash
```

`sudo` = "用管理员身份执行这个命令"。终端会让你输入密码（依然不显示字符），输完回车就行。

好，OpenClaw 主程序装好了。

但它现在还是一个"空壳"，知道自己是 OpenClaw，但不知道用哪个 AI 来思考、你的 API Key 是什么、该怎么跟你通信。

安装顺利的话，直接跳到第五章"通用配置篇"。如果遇到报错，先查阅下方 3.4 的常见问题排查。

3.4 Mac 常见问题排查——4 个最高频的坑

装的过程中遇到问题了？别慌。

我把 Mac 用户最常见的 5 个问题收集在这里。每 10 个找我求助的 Mac 用户里，有 7 个问题都在这 5 个里面。

问题 1：Homebrew 安装卡住——一直显示 Downloading 不动

原因：GitHub 服务器在国外，国内直连太慢。

解决：用国内镜像安装（前面已经给过了）：

```
/bin/zsh -c "$(curl-fsSLhttps://gitee.com/cunkai/HomebrewCN/raw/master/Homebrew.sh)"
```

如果安装进行了一半卡住，按 **Ctrl + C** 中断当前操作，然后用上面的国内镜像命令重新安装。

问题 2：安装脚本报 EACCES 权限错误

报错内容：**Error: EACCES: permission denied**

原因：当前用户没有权限往系统目录写入文件。

解决：在命令前加 **sudo**（管理员权限运行）：

```
sudo curl -fsSL https://clawd.org.cn/install.sh | bash
```

加 **sudo** 就是用管理员身份执行。输入 Mac 密码后回车（**密码不显示是正常的**）。

问题 3：`node -v` 显示版本低于 22

原因：之前安装过旧版 Node.js。

解决：

```
brew upgrade node
```

升级后再 **node -v** 验证。

如果升级后版本依然不对：

```
brew unlink node && brew link node --overwrite
```

如果还不行（极端情况）：

```
brew uninstall node && brew install node
```

完全卸载后重新安装。

问题 4：网关启动报"端口被占用"

报错内容：`Error: listen EADDRINUSE: address already in use :::18789`

原因：18789 端口已经被其他程序占用了。

解决：先查看谁占了这个端口：

```
lsof -i:18789
```

这行命令会显示占用该端口的进程名和 PID（进程 ID）。

然后杀掉该进程：


```
kill -9 <PID>
```

把 `<PID>` 替换成上面查到的数字。

杀掉后重新运行 `openclaw-cn gateway --port 18789`。

如果你不想杀进程，也可以在配置中修改 OpenClaw 的端口号：运行 `openclaw-cn configure`，在端口设置那一步输入一个不同的端口号（比如 18790）。

📄

好。Mac 安装篇全部结束。

接下来请跳到第五章“通用配置篇”，完成配置和首次运行。

如果你是 Windows 用户，请继续看下面的第四章。

四、Windows 部署篇——从零开始手把手教你

Windows 用户看这里。

首先，不要看上面的 Mac 部署篇。Mac 和 Windows 的操作完全不同，命令不一样，工具不一样，踩的坑也不一样。如果你参照 Mac 的步骤在 Windows 上操作，你一定会出问题。

Windows 确实比 Mac 多几个坑。这是事实。但这些坑全在我的掌控之中，我已经帮你把每个坑标好了位置，告诉你怎么跨过去。

跟着做就行。

4.1 第一步：安装 Node.js

Windows 安装 Node.js 不需要 Homebrew，直接从官网下载安装包，双击安装。

跟你平时装 QQ、装微信、装钉钉完全一样的流程。

Step 1: 下载安装包

打开浏览器，访问：<https://nodejs.org/en/download>

The screenshot shows the Node.js download page. At the top, it says "Download Node.js®". Below this, there are dropdown menus for "Get Node.js®" (set to v24.14.0 (LTS)), "for" (set to Windows), "using" (set to Docker), and "with" (set to npm). A green banner below these menus says "Info Want new features sooner? Get the latest Node.js version instead and try the latest improvements!". Below the banner is a dark box containing a list of instructions for installing Node.js using Docker. At the bottom of the page, there is a section "Or get a prebuilt Node.js® for" with dropdown menus for "Windows" and "running a" (set to x64), followed by the word "architecture.". Below this, there are two green buttons: "Windows Installer (.msi)" and "Standalone Binary (.zip)". A red arrow points to the "Windows Installer (.msi)" button with the text "②点击下载". Another red arrow points to the "x64" dropdown menu with the text "① 确认电脑系统架构".

Download Node.js®

Get Node.js® v24.14.0 (LTS) for Windows using Docker with npm

Info Want new features sooner? Get the latest Node.js version instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.14.0".
12
13 # Verify npm version:
14 npm -v # Should print "11.9.0".
```

PowerShell [Copy to clipboard](#)

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)

Or get a prebuilt Node.js® for Windows running a x64 architecture.

②点击下载 [Windows Installer \(.msi\)](#) [Standalone Binary \(.zip\)](#) ① 确认电脑系统架构

在页面上找到 "Windows Installer (.msi)"。

选择 64-bit 版本，现在的电脑基本都是 64 位的，你不需要特意确认。

（选看）如何确认自己电脑是 x64 还是 arm64 呢？

首先右键我的电脑->属性；然后就能看到系统类型中的标识，目前大部分 windows 电脑均为 **x64**



设备规格

设备名称	DESKTOP-XXXXXX
处理器	12th Gen Intel(R) Core(TM) i5-12400F 2.50 GHz
机带 RAM	16.0 GB (15.8 GB 可用)
设备 ID	XXXXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
产品 ID	XXXXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
系统类型	64 位操作系统, 基于 x64 的处理器
笔和触控	没有可用于此显示器的笔或触控输入

确保版本号 ≥ 22 。页面上应该会显示类似"v22.14.0 LTS"这样的字样。LTS 是"长期支持版"的意思，适合我们用。

点击下载。文件大概 100MB 左右。

Step 2: 运行安装包

下载完成后，找到那个 `.msi` 文件（通常在"下载"文件夹里），双击运行。

安装向导会弹出来。

接下来基本就是一路 Next，但有一个极其关键的地方你必须注意。

我拆开每一步给你看：

1. Welcome 页面：点 Next
2. License Agreement（许可协议）：勾选"I accept"，点 Next
3. Destination Folder（安装路径）：保持默认（`C:\Program Files\nodejs\`），点 Next
4. Custom Setup（自定义设置）：

在这个页面上，你会看到几个可选的组件。

确保 "Add to PATH" 这个选项是被勾选的（或者显示为"Will be installed on local hard drive"）。**没有这个勾，后面全白装。**

"Add to PATH"是什么意思？

简单说，它让 Windows 知道 `node` 和 `npm` 这两个命令在哪里。有了它，你在 PowerShell 里输入 `node`，系统能找到对应的程序。没有它，系统会告诉你"`node`不是内部或外部命令"。

一定要勾选。一定要勾选。一定要勾选。

重要的事情说三遍。

5. 继续 Next → Install（可能需要确认管理员权限）→ Finish

安装完成。

Step 3: 验证安装

按 `Win + R`（同时按住 Windows 键和 R 键），弹出"运行"对话框。

输入 `powershell`，回车。

PowerShell 窗口打开了。

在 PowerShell 里输入：

```
node -v
```

如果看到 `v22.x.x` 或更高的版本号，Node.js 安装成功。

再验证 npm：

```
npm -v
```

看到版本号就行。

如果提示"'node'不是内部或外部命令，也不是可运行的程序"，说明 PATH 没有正确配置。最快的解决方案：

1. 打开"控制面板 → 程序和功能"，找到 Node.js，卸载
2. 重新下载安装包，重新安装
3. 安装时仔细确认"Add to PATH"已勾选

卸载重装是最干净的办法。不要试图手动改 PATH，容易改出新问题。

好，Node.js 搞定了。跟装微信一样，下载、双击、一路 Next。Windows 的安装体验在这一步反而比 Mac 还省心。

4.2 第二步：安装 OpenClaw

Node.js 装好后，直接安装 OpenClaw。

Windows 的安装方式和 Mac 不同，用的是官方提供的 PowerShell 脚本，一行命令搞定。

极其重要，请以管理员身份打开 PowerShell。

方法：在 Windows 左下角的搜索栏里搜索"PowerShell"，然后右键点击搜索结果，选择"以管理员身份运行"。

打开后，窗口标题栏会显示"管理员: Windows PowerShell"，确认一下是管理员模式。

在 PowerShell 里输入：

```
iwr -useb https://clawd.org.cn/install.ps1|iex
```

拆解给你听：

- `iwr` → `Invoke-WebRequest` 的缩写，意思是：从这个 URL 下载内容
- `-useb` → 使用基本解析（Use Basic Parsing），兼容性更好
- `| iex` → 把下载的内容直接执行（Invoke-Expression）

说白了，就是从官方服务器下载安装脚本，然后立刻执行。跟 Mac 那边的 `curl | bash` 是同样的逻辑，只是 Windows 用的是 PowerShell 语法。

回车后等待。过程中看到黄色 WARN 不用管，只有红色 ERROR 才需要处理。

如果一切顺利，1-3 分钟后安装就完成了。

4.3 验证安装

安装脚本跑完后，验证一下：

```
openclaw-cn --version
```

如果看到版本号，你已经成功在 **Windows** 上安装了 **OpenClaw**。

如果你遇到了 `ExecutionPolicy` 报错：

报错可能长这样：无法加载文件 `xxxxxxx`，因为在此系统上禁止运行脚本。

或者英文：`cannot be loaded because running scripts is disabled on this system`

微软的安全策略，设计得像给你家大门上了三把锁，钥匙塞在锁孔里，你就是不知道转哪把。

解决方案，在 `PowerShell`（管理员模式）里执行：

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

这行命令把当前用户的脚本执行策略改为"`RemoteSigned`"，允许运行本地脚本和来自互联网的已签名脚本。

这是一个安全的设置，微软官方文档中也推荐这个策略。它只影响你当前用户，不影响系统其他用户。

执行后它可能会问你"确定要更改吗？"，输入 **Y 或者 A**，回车。

```
PS D:\openclaw> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
执行策略更改
执行策略可帮助你防止执行不信任的脚本。更改执行策略可能会产生安全风险，如 https://go.microsoft.com/fwlink/?LinkID=135170
中的 about_Execution_Policies
帮助主题所述。是否要更改执行策略？
[Y] 是(Y) [A] 全是(A) [N] 否(N) [L] 全否(L) [S] 暂停(S) [?] 帮助 (默认值为"N")： A
```

然后重新执行安装命令：

```
iwr -useb https://clawd.org.cn/install.ps1|iex
```

Windows 的坑确实比 Mac 多，但你全趟过来了。

能在 Windows 上把这一套跑通的人，说明动手能力已经超过了九成来问我私信的读者。真的。

接下来，跳到第五章"通用配置篇"，完成配置向导和首次运行。

4.4 Windows 常见问题排查——4 个最高频的坑

问题 1：ExecutionPolicy 报错——禁止运行脚本

前面已经讲过了。解决：

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

输入 Y 确认。然后重新执行安装脚本：

```
iwr -useb https://clawd.org.cn/install.ps1|iex
```

问题 2：`openclaw-cn` 命令找不到

报错：`openclaw-cn` 不是内部或外部命令

原因：安装脚本完成后，当前 PowerShell 窗口还没有刷新环境变量。

解决：关掉当前 PowerShell，以管理员身份重新打开一个新的窗口，再执行 `openclaw-cn --version` 验证。

问题 3：端口 18789 被占用

查看占用端口的进程：

```
netstat -ano | findstr 18789
```

最右边那一列是 PID（进程 ID）。

杀掉该进程：

```
taskkill /PID <PID 号> /F
```

把 **<PID 号>** 替换成查到的数字。/F 表示强制终止。

然后重新运行 `openclaw-cn gateway --port 18789`。

问题 4：防火墙拦截——WebUI 打不开

浏览器访问 `127.0.0.1:18789` 但页面加载不出来。

大概率是防火墙把 Node.js 的网络通信拦截了。

解决：

1. 打开"控制面板 → Windows Defender 防火墙 → 允许应用通过防火墙"
2. 点击"更改设置"
3. 找到列表中的 Node.js
4. 勾选"专用"和"公用"两个复选框
5. 确定

如果列表里找不到 Node.js，点击"允许其他应用"，手动添加 `C:\Program Files\nodejs\node.exe`。

好。Windows 安装篇全部结束。

接下来请跳到第五章"通用配置篇"。

五、通用配置篇——配置向导 + API Key + 首次运行

不管你是 Mac 用户还是 Windows 用户，走到这一步，说明你已经成功安装了 OpenClaw 主程序。

接下来的配置和启动步骤，Mac 和 Windows 完全一样。

5.1 运行配置向导

在终端（Mac）或 PowerShell（Windows，管理员模式）输入：

```
openclaw-cn onboard --install-daemon
```

这行在干什么？ `openclaw-cn onboard` 启动初始化配置向导（`onboard` = "入职引导"，就像新员工入职培训）；`--install-daemon` 顺便把守护进程装上（让 OpenClaw 能在后台持续运行）。

回车后，配置向导就启动了。

```
PS D:\openclaw> openclaw-cn onboard

OpenClaw-CN 0.1.7 (415f7b6) - 你的消息，你的服务器，Meta的眼泪。

检测到 Windows 系统。
强烈推荐使用 WSL2；原生 Windows 未经测试且问题较多。
指南: https://docs.clawd.bot/windows

CLAWD.BOT
  每日新鲜

OpenClaw 安装引导

安全
安全警告 - 请阅读。

OpenClaw 是一个业余项目，仍处于测试阶段。可能会有粗糙之处。
如果启用了工具，此机器人可以读取文件并执行操作。
不良提示可能会诱使其执行不安全的操作。

如果您不熟悉基本的安全和访问控制，请不要运行 OpenClaw。
在启用工具或将系统暴露给互联网之前，请寻求有经验的人士帮助。

推荐的基础设置：
- 配对/白名单 + @提及门控。
- 沙箱 + 最低权限工具。
- 将机密信息保留在智能体可访问的文件系统之外。
- 对于使用工具或不受信任收件箱的机器人，请使用最强大的可用模型。

定期运行：
openclaw-cn security audit --deep
openclaw-cn security audit --fix

必读: https://clawd.org.cn/gateway/security.html
```

当你看到这个界面，恭喜你，你已经成功安装 Openclaw 并且超越了 70%的人了，后面就是简单的配置过程了

它不是一下子让你填完所有东西，而是一步一步问你问题，你根据我下面的指引逐步回答就行。

如果某一步你不确定选什么，选默认的。默认值是开发团队推荐的配置，适合大多数人。后面随时可以改。

我把每一步都详细拆开讲：

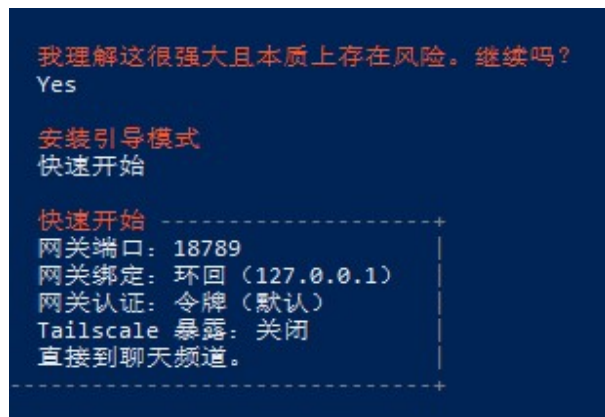
向导第 1 步：选择工作区路径

配置向导首先会问你：把 OpenClaw 的数据（配置文件、聊天记录、技能等）存在哪个目录里？

它会给你一个默认路径，直接回车，用默认就行。

除非你有特殊需求（比如想把数据放在外接硬盘上），否则不需要改。

回车，进入下一步。



向导第 2 步：选择 AI 模型提供商

这一步非常关键，它决定了你的 OpenClaw 用哪个"大脑"来思考。

向导会给你列出一堆选项：

- Anthropic（Claude 系列，效果最好，但需要代理+付费）
- OpenAI（GPT 系列，老牌选手，需要代理+付费）
- DashScope/阿里云（通义千问，国内常用，免费额度充足）
- Moonshot（Kimi，国内替代方案，长文本很强）

- **MiniMax**（国内模型，**Coding Plan** 专为高频调用设计，单价低、用量足）
- Google（Gemini，需要代理）
- Cloudflare AI
- Z.AI
- 其他



选项很多，别懵。

如果你是国内用户，我只推荐一个选择：**MiniMax（Coding Plan）**。

为什么？三个原因：

1. **OpenClaw 是 Agent, Token 消耗远大于普通聊天**, Coding Plan 是专为开发者/高频调用场景定价的套餐, **单价低、用量足, 日常跑任务基本够用**, 不像按需计费那样动不动就心疼
2. 不需要代理/翻墙, 国内网络直连, 响应速度很快
3. 效果足够好, 日常对话、任务执行、文件处理都没问题, 模型持续在迭代升级
4. **最重要的原因: 目前新用户会赠送 15 元的体验金, 大家可以先直观体验 Openclaw 的应用, 然后再决定是否需要升级套餐或者更换其他国外厂商的模型**

Claude 效果确实更好, 但你得先有代理, 还得付费, 而且 Agent 模式下 Claude 的调用费用会相当高。在你对 OpenClaw 还不够熟悉的阶段, 先用性价比高的跑起来, 比什么都重要。

等你用了一段时间, 觉得 MiniMax 满足不了你了, 再换 Claude 也来得及。

用方向键上下选择 "MiniMax" 相关的选项, 然后回车。

向导第 3 步: 输入 API Key

好, 到了最让人紧张的一步——输入 API Key。

"我还没有 API Key 啊！"

别急。跟着下面的步骤, 5 分钟就能搞到。

【MiniMax Coding Plan API Key 获取完整教程——手把手级别】

方法一: 选择第二个 API key


```
MiniMax 认证方法
MiniMax OAuth
> MiniMax API key (M2.5 / M2.1 / M2)
返回
```

Step 1: 打开浏览器（Safari/Chrome/Edge 都行），在地址栏输入：

<https://platform.minimaxi.com/>

这是 MiniMax 的开发者平台，也是获取和管理 API Key 的地方。

Step 2: 如果你没有 MiniMax 账号，点击页面右上角的"注册"按钮。

注册方式很简单，手机号 + 验证码，完成实名认证后即可使用。整个流程不超过 3 分钟。

Step 3: 登录后，在页面左侧菜单找到"API Keys"入口（或点击右上角头像 → 账号设置 → API Keys）。



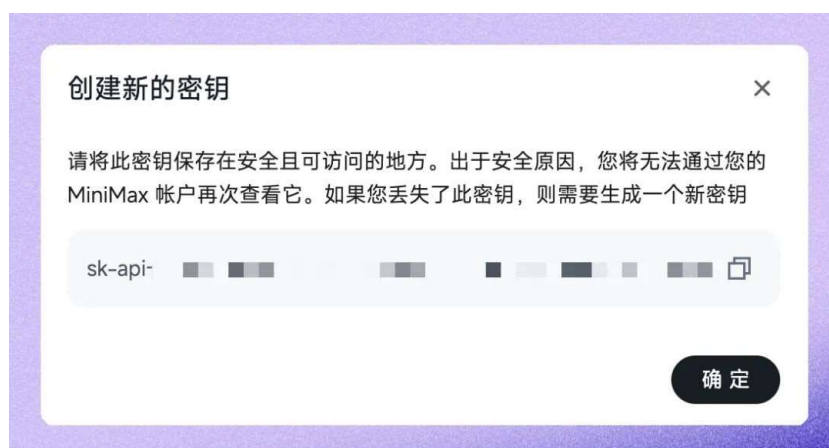
点击进入。

Step 4: 点击页面上的"创建新的 API Key"或"+ 新建"按钮。

在弹出的对话框里给这个 Key 取个名字（比如"OpenClaw"），方便你以后认出它是干嘛用的，然后点击确认。

关于 **Coding Plan**：MiniMax 面向开发者推出的 Coding Plan 套餐，**专为 Agent 这类高频调用场景设计**，相比按需计费单价更低、用量更足。OpenClaw 每次执行任务要来回调用好几轮模型，用 Coding Plan 跑日常任务基本够用，不会动不动就超额。进入控制台后可以在"用量与计费"页面查看实时消耗。

Step 5: 系统会生成一串类似这样的字符串：

[illegible]

这串字符就是你的 **API Key**。你的"会员卡号"。

极其重要的两件事：

1. **这个 Key 只会完整显示一次。** 页面关掉后你就再也看不到了（只能看到最后几位）。所以，现在立刻复制下来，粘贴到一个安全的地方。备忘录、记事本、密码管理器都行。

2. **永远不要把 API Key 发给别人、贴到朋友圈、发到群里。** 它就像你的银行卡密码，别人拿到了就能用你的额度。如果你怀疑 Key 泄露了，立刻回到 MiniMax 平台删掉它并创建一个新的。

好，Key 拿到手了。

回到终端的配置向导，它正在等你输入 API Key。

在终端里粘贴 Key：

- Mac 终端的粘贴快捷键：**Command + V**

- Windows PowerShell 的粘贴方式：右键单击窗口（传统方式）或 **Ctrl + V**（较新版本支持）

粘贴后你可能看不到完整的 Key 内容（出于安全考虑，终端不会显示你输入的敏感信息）。这是正常的。

直接按回车。

如果 Key 没问题，向导会自动进入下一步。

如果提示 Key 无效，检查一下是不是复制的时候多了空格或者少了字符。重新复制粘贴一次。

方法二（推荐）：

选择第一个 MiniMax OAuth 方法

```
MiniMax 认证方法
> MiniMax OAuth (OAuth plugin for MiniMax)
MiniMax API key
返回
```

选择第二个

国内域名的 Minimax

```
Select MiniMax endpoint
Global
> CN (OAuth for users in China)
```

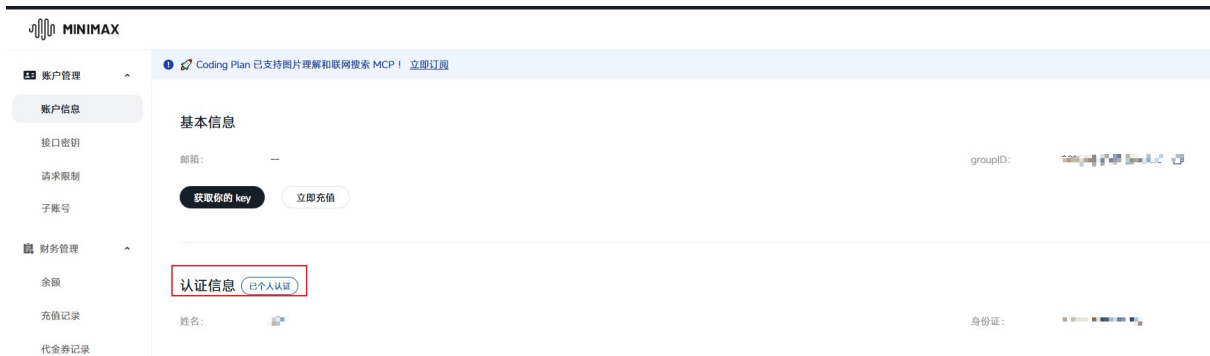
然后稍等一段时间，会弹出页面请求你授权登录，点击“授权”



然后直接键入 enter，代表所有模型均使用即可配置完成

```
*/model选择器中的模型（多选）
[+] minimax-portal/MiniMax-M2 (MiniMax M2 · ctx 200k · alias: minimax-m2)
[+] minimax-portal/MiniMax-M2.1 (MiniMax M2.1 · ctx 200k · alias: minimax-m2.1)
[+] minimax-portal/MiniMax-M2.1-highspeed (MiniMax M2.1 Highspeed · ctx 200k · alias: minimax-m2.1-highspeed)
[+] minimax-portal/MiniMax-M2.5 (MiniMax M2.5 · ctx 200k · reasoning · alias: minimax-m2.5)
[+] minimax-portal/MiniMax-M2.5-highspeed (MiniMax M2.5 Highspeed · ctx 200k · reasoning · alias: minimax-m2.5-highspeed)
```

注意：15 元的体验金需要个人认证才会到账，可以在这里认证



然后点击余额页面，如果显示下图代表到账成功



向导第 4 步：配置网关端口

它会问你：OpenClaw 网关使用哪个端口？

默认是 18789。直接回车。

端口就像门牌号，你家住在"18789 号"，别的程序住在别的号。只要不跟其他程序冲突就行，而 18789 基本没人用。

向导第 5 步：通信通道配置

它会问你要不要配置聊天平台的接入（飞书、Telegram 等）。

现在选"跳过（Skip）"。

我们先把 OpenClaw 本体跑通。聊天平台的接入放到后面第六章进阶篇再讲。

一步一步来。别贪多。一次想配太多东西，出了问题你都不知道是哪一步出的。

向导第 6 步：是否安装守护进程

它会问你：Install daemon? (Y/n)

输入 Y，回车。

守护进程是什么？就像你手机上的微信后台运行，即使你没有打开微信 App，微信也在后台跑着。有新消息了它能第一时间收到并推送给你。

OpenClaw 的守护进程也是这样，安装后，OpenClaw 会在后台持续运行。你不需要每次用的时候都手动启动它。

在 Mac 上，守护进程是通过系统自带的 `LaunchAgent` 实现的，非常轻量，几乎不占用 CPU 和内存。在 Windows 上，可能通过"计划任务（Task Scheduler）"或者"Windows 服务"来实现。如果安装过程中提示需要额外确认，按提示操作。

如果守护进程安装失败了也没关系，最多每次用 `OpenClaw` 的时候手动运行 `openclaw-cn gateway --port 18789`。功能完全不受影响，只是少了"开机自启"的便利性。

向导第 7 步：确认配置摘要

最后，向导会给你显示一个配置摘要，你选了什么模型、工作区路径在哪、端口号是多少、守护进程是否安装。

过一眼确认没问题，然后回车。

当你看到"Configuration complete!"（配置完成）的提示——

深呼吸。配置向导跑完了。

你离"跟 AI 对话"这个终极目标，只剩最后一步了。

Windows 用户注意——防火墙弹窗：

当你启动网关时（或者配置过程中），Windows 防火墙很可能会弹出一个弹窗，问你是否允许 `Node.js` 通过网络通信。

点"允许访问"。

如果你手快点了"取消"，OpenClaw 的网关可能无法正常工作（因为它需要在本地网络上监听端口）。

补救方法：打开"控制面板 → Windows Defender 防火墙 → 允许应用通过防火墙"，找到 Node.js 相关的项目，勾选"专用网络"和"公用网络"。

5.2 首次运行——见证奇迹的时刻

好。

所有准备工作都完成了。

Node.js 装了。OpenClaw 装了。配置向导跑完了。API Key 填好了。

现在，按下那个按钮，让引擎发动。

在终端/PowerShell 输入：

```
openclaw-cn gateway --port 18789
```

这个你已经懂了：启动 OpenClaw 的网关服务，负责接待你发来的消息，把消息转发给 AI 大模型处理，再把 AI 的回复传回给你。--port 18789 是指定端口号，用默认值就行。

回车后，你会看到终端里刷出一堆日志信息。各种 INFO、时间戳、模块名之类的。

不用看懂。

只需要关注一个关键信息：当你看到类似以下的字样时——

```
Gateway is running on port 18789
```

或者

```
Server started at http://127.0.0.1:18789
```

这说明，OpenClaw 已经在运行了。

不确定网关是否在跑？另开一个终端窗口，输入：

```
openclaw-cn gateway status
```

它会告诉你当前网关的运行状态。

现在，不要关闭终端窗口（关了的话网关就停了，除非你安装了守护进程）。

打开浏览器，或者直接输入命令：

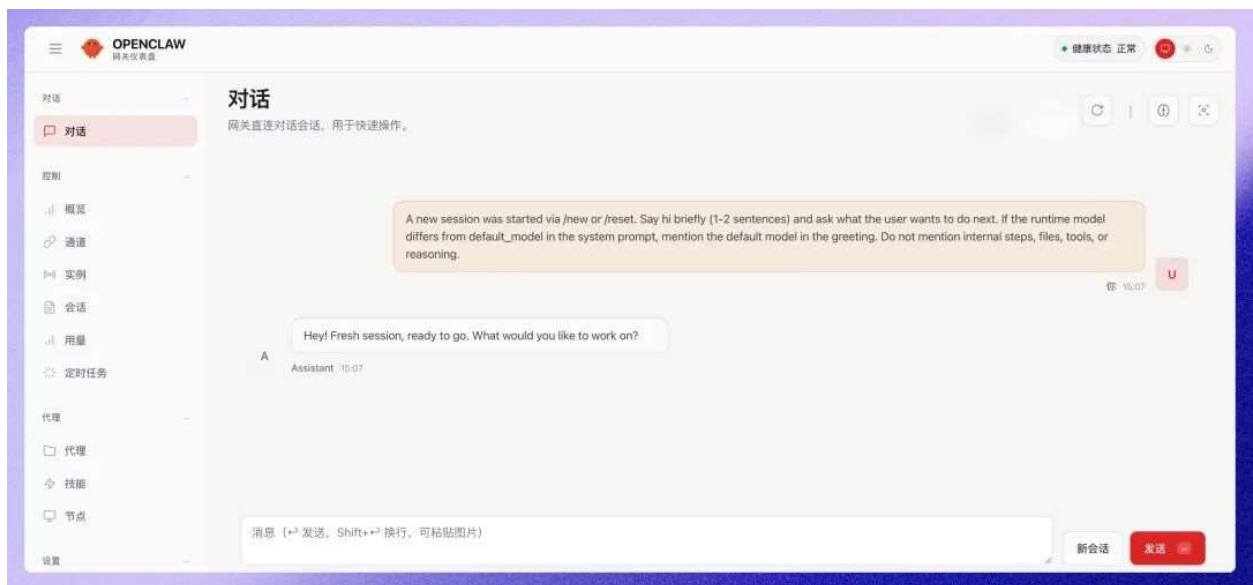
openclaw-cn dashboard

它会自动打开浏览器并跳转到控制台。你也可以手动在地址栏输入：

http://127.0.0.1:18789/

127.0.0.1 就是"你自己的电脑"的意思。:18789 是端口号。

回车。



如果你看到了一个干净的聊天界面，一个输入框，等待你说第一句话——

现在，在输入框里打字：

你好，介绍一下你自己

回车。

等几秒钟



AI 回复了。它在跟你说话。它在向你介绍自己。

这不是一个网页上的聊天工具。这是一个跑在你自己电脑上的 AI 代理。

你的数据不会离开你的设备。 你的 API Key 只有你自己知道。你拥有完全的控制权。

你成功了。从复制粘贴第一行命令到这一刻，你完成了一件 23 万人给它点 Star 的事。

认真的。

你知道有多少人在 Homebrew 那一步就放弃了吗？有多少人在 API Key 那一步就懵了吗？有多少人折腾了一周最后放弃了吗？

但你没有。

你一行命令一行命令地敲过来了。每一个报错你都没有被吓跑。每一个陌生的概念你都耐心看完了。

从复制粘贴第一行命令，到听见 AI 第一句回复——这 30 分钟，你跨过了 23 万人给它点 Star 之前的那道门槛。

好，如果你想先休息一下，完全可以。OpenClaw 已经跑起来了。如果你装了守护进程，关掉终端也没关系，下次直接运行 `openclaw-cn dashboard` 或手动打开浏览器访问 <http://127.0.0.1:18789/> 就行。

5.3 配置和运行常见问题

问题：API Key 配置了但对话没有回复 / 回复报错

可能原因 1：API Key 输错了

运行 `openclaw-cn configure`，重新输入 Key。复制的时候注意不要多复制了空格或者换行符。

可能原因 2：网络问题（如果用的是海外模型）

Claude、GPT 等模型的 API 在国外，国内直连不通。要么配代理（后面进阶篇讲），要么换国内模型。

可能原因 3: Token 用量超额了

登录 MiniMax 开发者平台（<https://platform.minimaxi.com/>），在"用量与计费"页面查看你的 API 调用量和剩余 Token 额度。OpenClaw 是 Agent，每次任务会多轮调用，消耗比普通聊天快——这正是推荐 Coding Plan 的原因，它的单价更低。

可能原因 4: 模型服务暂时不可用

偶尔 MiniMax 的服务会短暂波动。等几分钟重试，或者查看 MiniMax 官方状态页面确认服务是否正常。

如果以上都不是，运行 `openclaw-cn doctor` 让 OpenClaw 自我诊断一下，它会告诉你哪里出了问题。

六、进阶配置篇——从"能用"到"好用"到"玩出花"

OpenClaw 装好了。能聊天了。能执行简单任务了。

但说实话，你现在只发挥了它不到 20% 的能力。

就像你买了一辆特斯拉，但只会用它代步，还没开过自动驾驶、没试过狂暴模式、没连过手机蓝牙。

接下来这一章，我带你把剩下的 80% 也解锁了。

6.1 切换 AI 模型——找到最适合你的"大脑"

你在配置向导里选了 MiniMax？很好。它够用。能完成大部分日常任务。

但 OpenClaw 支持十几个模型提供商。每个模型的风格、能力、价格都不一样。就像同一款手机可以插不同运营商的 SIM 卡，换模型就像换 SIM 卡，手机还是那个手机，但信号源变了，体验可能完全不同。

想换模型？很简单。在终端/PowerShell 里输入：

```
openclaw-cn configure
```

这会重新启动配置向导。你可以选择新的模型提供商，输入新的 API Key。其他配置不需要改。

主流模型提供商对比，帮你做选择：

模型提供商	效果	国内可用	Agent 场景性价比	价格	适合谁
MiniMax (Coding Plan)	★★★★	是	★★★★★ 高频调用 省钱	低	国内用户入门首选

模型提供商	效果	国内可用	Agent 场景性价比	价格	适合谁
通义千问	★★★★	是	★★★★	低	国内用户备选
Moonshot/Kimi	★★★★	是	有	适中	需要处理长文本的用户
Claude	★★★★★	需代理	少量	较贵	追求最佳效果的进阶用户
OpenAI GPT	★★★★☆	需代理	少量	较贵	编程和推理任务
Gemini	★★★★☆	需代理	有	适中	多模态任务（图片理解等）

我的推荐路径：

1. 起步阶段：MiniMax Coding Plan。OpenClaw 是 Agent，Token 消耗比普通聊天大，Coding Plan 专为高频调用定价，国内直连，跑日常任务基本够用、不心疼。且新用户有 15 元免费体验金，可以愉快的玩耍

2. 进阶阶段：如果你有代理，换 Claude 试试。Claude 在理解复杂任务和执行多步骤操作方面明显强于其他模型。OpenClaw 2026.2.23 版本已经支持最新的 Claude Opus 4.6。
3. 日常使用：可以根据任务类型切换，简单任务用 MiniMax（省钱），复杂任务切 Claude（效果好）。

各模型 API Key 获取地址汇总：

- MiniMax: <https://platform.minimaxi.com/>
- 通义千问: <https://dashscope.console.aliyun.com/>
- Kimi/Moonshot: <https://platform.moonshot.cn/>
- Claude: <https://console.anthropic.com/>
- OpenAI: <https://platform.openai.com/>
- Gemini: <https://aistudio.google.com/>

流程都差不多，注册账号、创建 API Key、复制粘贴到 OpenClaw 里。

6.2 代理/网络配置——国内用户的特殊处理

如果你用的是国内模型（MiniMax、通义千问、Kimi 等），这一节可以跳过。国内模型不需要代理。

如果你想用 Claude、GPT、Gemini 这些国外模型，你需要配置代理。

为什么？因为这些模型的 API 服务器在国外，国内网络直连不上。就像你用国内手机号打美国电话，不开国际漫游就打不通。

配置方式取决于你用的代理工具。大部分代理工具都会提供一个本地 HTTP 代理地址，通常长这样：<http://127.0.0.1:7890>（端口号可能是 7890、1080、8080 等，看你的工具设置）。

Mac 设置方式：

```
export https_proxy=http://127.0.0.1:7890 export http_proxy=http://127.0.0.1:7890 export all_proxy=http://127.0.0.1:7890
```

Windows PowerShell 设置方式：

```
$env:https_proxy="http://127.0.0.1:7890" $env:http_proxy="http://127.0.0.1:7890" $env:all_proxy="http://127.0.0.1:7890"
```

注意：[7890](#) 是你代理工具的端口号，不同工具不一样。打开你的代理工具设置页面，找到"HTTP 代理端口"或"本地端口"那个数字，替换进去。

设置好后，运行 `openclaw-cn gateway --port 18789`。

另一个方案是在 OpenClaw 的配置文件中永久设置代理，这样你不用每次启动前都手动设环境变量。具体方法参考 OpenClaw 中文社区文档。

嫌麻烦？

确实麻烦。2026 年了，用个 API 还得配代理，这体验约等于用传真机发邮件。代理配置是很多人放弃使用海外模型的原因。

我的建议：**如果你不想折腾代理，直接用国内模型就完事了。**

MiniMax Coding Plan 是专门为高频调用设计的套餐，单价低，跑 OpenClaw 这类 Agent 基本够用。Kimi 在处理长文本方面甚至比很多国外模型还强。新用户认证赠送 15 元体验金

别让代理成为你使用 OpenClaw 的拦路虎。先用起来。体验了 OpenClaw 的强大之后，你自然会有动力去折腾代理。

6.3 接入聊天平台——解锁真正的“随时随地”

前面你一直是通过浏览器的 WebUI 跟 OpenClaw 对话的。

这已经很不错了。

但 OpenClaw 的真正杀手锏是，**你可以把它接入飞书、Telegram、Discord 等聊天平台。**

装好之后，你可以躺在沙发上，用手机上的飞书对它说“帮我整理桌面上的 PDF”，它就真的去整理了。

不需要打开电脑。不需要打开浏览器。不需要坐在桌前。

你发一条飞书消息，OpenClaw 在你电脑后台默默干活。干完了给你发一条消息：“搞定了，文件已按日期归档到 Documents/Archive/文件夹下。”

这才是数字管家应有的样子。

这里我重点讲飞书接入，国内用户最常用、OpenClaw 支持最好的平台。

飞书接入分两部分：先在飞书开放平台创建应用（7 步），再在 OpenClaw 里配置连接（5 分钟）。全程跟着走，不需要任何开发经验。

第一部分：在飞书开放平台创建自建应用

Step 1：打开飞书开放平台

打开浏览器，访问：<https://open.feishu.cn/app>

用你的飞书账号登录（扫码或密码均可）。

Step 2：创建企业自建应用

登录后，点击右上角"创建应用"按钮，选择"企业自建应用"。



在弹出的表单里填写：

- **应用名称**：随便取，比如"我的 AI 助手"或"OpenClaw"
- **应用描述**：随便写几个字，比如"个人 AI 助手"
- **应用图标**：可以跳过，默认图标就行

填完点"创建"，进入应用详情页面。

Step 3: 获取 App ID 和 App Secret

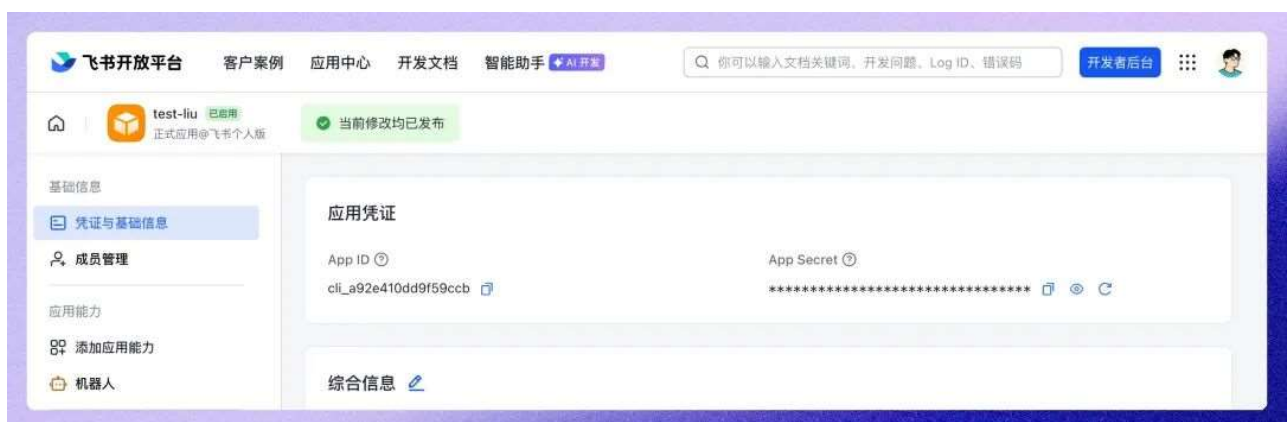
进入应用后，在左侧菜单找到"凭证与基础信息"，点击进入。

你会看到两个关键信息：

- **App ID**：格式是 `cli_xxxxxxxxxxxxxx`（以 `cli_` 开头）
- **App Secret**：一串随机字符，默认隐藏，点击"查看"后复制

把这两个值复制下来存好，后面配置 **OpenClaw** 要用。

和 API Key 一样，App Secret 属于机密信息，**永远不要发给别人**。



Step 4: 配置权限

在左侧菜单点击"权限管理"。

这一步告诉飞书：你的机器人被允许做什么。

OpenClaw 需要以下核心权限——点击"批量申请权限"，搜索并勾选：



```
{
  "scopes": {
    "tenant": [
      "aily:file:read",
      "aily:file:write",
      "application:application.app_message_stats.overview:readonly",
      "application:application:self_manage",
      "application:bot.menu:write",
      "cardkit:card:write",
      "contact:contact.base:readonly",
      "contact:user.employee_id:readonly",
      "corehr:file:download",
      "docs:document.content:read",
      "event:ip_list",
      "im:chat",
      "im:chat.access_event.bot_p2p_chat:read",
      "im:chat.members:bot_access",
      "im:message",
      "im:message.group_at_msg:readonly",
      "im:message.group_msg",
      "im:message.p2p_msg:readonly",
      "im:message:readonly",
      "im:message:send_as_bot",
      "im:resource",
      "sheets:spreadsheet",
      "wiki:wiki:readonly"
    ],
    "user": [
      "aily:file:read",
      "aily:file:write",
      "im:chat.access_event.bot_p2p_chat:read"
    ]
  }
}
```

确认导入权限

应用身份权限

用户身份权限

▼ 本次新增权限 (23)

☐ 获取文件

☐ 获取机器人发送消息的概览数据

☐ 获取通讯录基本信息

☐ 查看云文档内容

☐ 订阅访问机器人会话相关事件

☐ 获取单聊、群组消息

☐ 获取群组中所有消息 (敏感权限)

☐ 查看、评论、编辑和管理电子表格

☐ 上传文件

☐ 创建、更新、删除机器人菜单

☐ 获取用户 user ID

☐ 获取事件的出口 IP

☐ 订阅机器人进、出群事件

☐ 以应用的身份发消息

☐ 读取用户发给机器人的单聊消息

☐ 查看知识库

☐ 管理应用自身资源

☐ 创建与更新卡片

☐ 下载文件

☐ 获取与更新群组信息

☐ 获取与发送单聊、群组消息

☐ 接收群聊中@机器人消息事件

☐ 获取与上传图片或文件资源

取消

上一步，导入权限

申请开通

勾选后提交。

个人飞书账号的权限通常会立即生效。企业账号可能需要管理员审批，提交后等通知即可。

Step 5: 启用机器人能力

在左侧菜单找到"应用能力"，点击进入，找到"机器人"这一项，点击"启用机器人"。

启用后，你可以给机器人设置一个显示名称，这会在飞书聊天界面显示为机器人的昵称。



第二部分（插在这里）：先在 OpenClaw 中配置飞书渠道并启动网关

⚠ 顺序很重要！在飞书开放平台配置事件订阅（Step 6）之前，必须先完成以下两步：1. 在 OpenClaw 里添加飞书渠道 2. 确认 OpenClaw 网关正在运行如果网关没跑起来，飞书的长连接设置会保存失败，机器人永远收不到消息。

方式一：通过配置向导（推荐）

在终端（Mac）或 PowerShell（Windows）输入：

```
openclaw-cn channels add
```

或者重新运行配置向导：

```
openclaw-cn onboard
```


向导会一步步引导你：选择平台（选"飞书"）→ 输入 App ID → 输入 App Secret。
填完就行。

```
(Did you mean channels?)
PS D:\openclaw> openclaw-cn channels add

@ OpenClaw-CN 0.1.7 (415f7b6) - 适用于Android。疯狂的概念，我们知道。

06:18:06          feishu_calendar: Registered feishu_calendar tool (8 actions)
[feishu] plugins registered: 9 modules loaded
T  通道设置
|
| o 通道状态 -----+
| | Feishu: needs credentials |
| | Telegram: 未配置          |
| | WhatsApp: 未配置          |
| | Discord: 未配置           |
| | Google Chat: 未配置       |
| | Slack: 未配置             |
| | Signal: 未配置            |
| | iMessage: 未配置          |
| |-----+
|
| o 现在配置聊天通道吗?
| | Yes
|
| o 通道工作原理 -----+
| | DM安全: 默认为配对: 未知的私信会获得配对码。
| | 批准命令: openclaw-cn pairing approve <channel> <code>
| | 公开私信需要 dmPolicy="open" + allowFrom=["*"].
| | 多用户私信: 设置 session.dmScope="per-channel-peer" 来隔离会话。
| | 文档: start/pairing
| |
| | Telegram: 最简单的入门方式—使用 @BotFather 注册一个机器人并开始使用。
| | WhatsApp: 使用您自己的号码工作: 建议使用独立手机 + eSIM。
| | Discord: 目前支持非常好。
| | Google Chat: 带有HTTP webhook的Google Workspace聊天应用。
| | Feishu: 飞书/Lark 机器人集成 (WebSocket)。
| | Slack: 已支持 (Socket模式)。
| | Signal: signal-cli链接设备: 更多设置 (David Reagans说: "加入Discord吧。")。
| | iMessage: 这仍然是一个正在进行的工作。
| |-----+
|
| * 选择一个通道
| | Telegram (Bot API)
| | WhatsApp (QR link)
| | Discord (Bot API)
| | Google Chat (Chat API)
| | > Feishu/Lark (飞书) (requires Lark Open Platform app)
| | Slack (Socket Mode)
| | Signal (signal-cli)
| | iMessage (imsg)
| | 已完成
| |-----+
```

```
Feishu account
Add a new account

New Feishu account id
test

选择平台 / Select platform
飞书 (国内版)

输入 飞书 App ID (cli_...)
XXXXXXXXXXXX

输入 飞书 App Secret
XXXXXXXXXXXXXXXXXXXXXXXXXXXX

选择一个通道
已完成

已选通道 -----
Feishu - 飞书/Lark enterprise messaging. 文档:
feishu |
```

方式二：直接编辑配置文件

如果你更喜欢手动操作，也可以直接编辑配置文件。

配置文件路径：`~/.openclaw/openclaw.json`

用文本编辑器打开（Mac 用 TextEdit，Windows 用记事本），在 `channels` 里加入以下内容：

```
{
  "channels": {
    "feishu": {
      "enabled": true,
      "dmPolicy": "pairing",
      "accounts": {
        "main": {
          "appId": "cli_你的 AppID",
          "appSecret": "你的 AppSecret",
```

```
        "botName": "我的 AI 助手"
    }
}
}
```

把 `cli_` 你的 `AppID` 和 你的 `AppSecret` 替换成 Step 3 里复制的实际值，`botName` 填机器人名字，保存文件。

启动网关，确认运行状态

保存配置后，启动（或重启）OpenClaw 网关：

```
openclaw-cn gateway --port 18789
```

或者

```
openclaw-cn gateway restart
```

验证网关正在运行：

```
openclaw-cn gateway status
```

看到"running"状态，说明 OpenClaw 已经就位。**确认运行后，回到飞书开放平台继续 Step 6。**

Step 6（回到飞书）：配置事件订阅

在左侧菜单点击"事件订阅"。

接收事件的方式，选"使用长连接接收事件"（WebSocket 模式）。

事件配置回调配置加密策略

事件配置

订阅事件后，飞书开放平台将会在事件（如机器人入群）发生时向请求地址推送消息[了解更多](#)

订阅方式

☒ 使用长连接接收事件推荐

☐ 将事件发送至开发者服务器

配置方式

无需注册公网域名或配置加密策略，仅需使用[官方 SDK](#) 启动长连接飞书客户端，并确保连接成功后，即可开启该模式

保存

取消

为什么是长连接，不是 URL 回调？

URL 回调需要你有一个公网 IP 或域名，本地部署的机器基本没有这个条件。长连接是 OpenClaw 主动去飞书服务器拉消息，不需要公网 IP，在家里或公司网络下都能用。**对于本地部署，选长连接就对了。**

选好之后，在下方"添加事件"里搜索并添加：

```
im.message.receive_v1
```

添加事件

Q

im.message.receive_v1

应用身份订阅(1)

用户身份订阅(0)

接收消息 v2.0

im.message.receive_v1

用户发送新消息至机器人或群聊后推送事件

这个事件的作用是：每当你给机器人发消息，飞书就会通知 OpenClaw 来处理。

保存后，飞书会和你本地正在运行的 OpenClaw 网关建立长连接，确认连通。这也是刚才必须先把网关跑起来的原因。



Step 7: 发布应用

在左侧菜单点击"版本管理与发布"。

点"创建版本"，填写版本号（比如 **1.0.0**）和版本说明（随便写），然后点"申请发布"。

应用不发布，机器人就在飞书里搜索不到。这一步不能跳过。

个人飞书账号通常无需审核，直接发布即可。企业账号需要管理员审批，提交后等通过就行。

两边都配置完毕。

点击打开应用：

14:24



开发者小助手 机器人 ▶



在飞书开放平台，企业开发者可通过丰富的 API 和 SDK 开...

今天

14:24



应用审批通过，已发布成功

你的应用“Openclaw”已通过管理员审批并发布成功！

打开应用

[自建应用发布申请] 自动审核通过

@龚家政 修改并提交了“Openclaw”，基于免审规则，系统已自动审核通过。

查看审核详情

配置免审规则



≡ 应用开发

≡ 开发交流

≡ 开发资源

验证：发送第一条消息

打开飞书（App 或网页版），在搜索栏里搜索你在 Step 2 填的应用名称。

找到机器人，发一条消息：

你好



机器人回复了吗？

恭喜，飞书接入成功。你现在可以躺在沙发上用手机发指令，让 **OpenClaw** 在电脑后台默默干活了。

进阶：访问控制配置

默认情况下，第一次给机器人发消息的用户需要"配对"才能使用，这是安全机制，防止别人乱用你的机器人（毕竟 API 消耗是你的）。

查看待批准的配对请求：

```
openclaw-cn pairing list feishu
```

批准某个用户：

```
openclaw-cn pairing approve feishu <配对码>
```

```
PS C:\Windows\system32> openclaw-cn pairing approve feishu E6RCMVT4
❖OpenClaw-CN 0.1.7 (415f7b6) — 我基本上是一把瑞士军刀，但有更多的观点和更少的锋利边缘。
Approved feishu sender [redacted]
PS C:\Windows\system32>
```

如果是完全自用，嫌每次审批麻烦，可以把配置文件里的 `dmPolicy` 改成 `"open"`。如果想限制只有特定人能用，改成 `"allowlist"`。



你好

1 条回复



你好

你好小政！👋
很高兴在飞书收到你的消息！有什么我能帮你的吗？

飞书接入常见问题

问题 1：机器人收不到我的消息

按顺序检查这 5 点：

1. 应用有没有发布？（Step 7）没发布的应用，机器人不会响应任何消息。
2. 事件订阅里有没有配置 `im.message.receive_v1`？（Step 6）
3. 接收方式是不是选的"长连接"，不是 URL 回调？（Step 6）
4. `im:message` 权限有没有通过审核？（Step 4）
5. OpenClaw 网关有没有在运行？运行 `openclaw-cn gateway status` 检查。

5 项都没问题？再跑一下诊断：

```
openclaw-cn logs --follow
```

看实时日志，找红色 ERROR 信息。

问题 2：在群里@机器人没反应

群组消息需要额外的权限 `im:message.group_msg`（Step 4 里提到的敏感权限）。
去飞书开放平台 → 权限管理，添加这个权限。

另外，群组里默认需要@机器人才触发响应（@你的机器人名字 + 消息内容）。
如果群组里希望不@也能响应，参考官方文档的 `requireMention: false` 配置。

问题 3：App Secret 泄露了怎么办

立刻去飞书开放平台，进入你的应用 → 凭证与基础信息 → 重置 App Secret。

重置后旧 Secret 立刻失效，新 Secret 立刻生效。

然后更新 OpenClaw 配置文件里的 `appSecret`，重启网关。

问题 4：发消息失败，提示权限不足

检查 `im:message:send_as_bot` 权限是否已申请通过（Step 4）。这是机器人发送消息的必要权限，缺了就只能收不能回。

除了飞书，OpenClaw 还支持 20 多个聊天平台。列几个常用的：

- 国内：飞书、钉钉、企业微信
- 国际：Telegram、Discord、Slack、WhatsApp
- Apple 生态：iMessage
- 其他：Line、Matrix、Signal、Messenger 等

每个平台的接入方式类似，安装对应的插件 → 在平台方创建应用/机器人 → 获取 Token → 配置到 OpenClaw → 完成。

6.4 技能（Skills）扩展——让 OpenClaw 学会新本事

OpenClaw 还有一个非常酷的机制，**Skills（技能）**。

Skills 是预制的能力包。安装一个 Skill，OpenClaw 就多学会一项新技能。

就像你的手机出厂只有打电话发短信的功能，但通过安装 App 你可以让它变成导航仪、计算器、相机、音乐播放器。

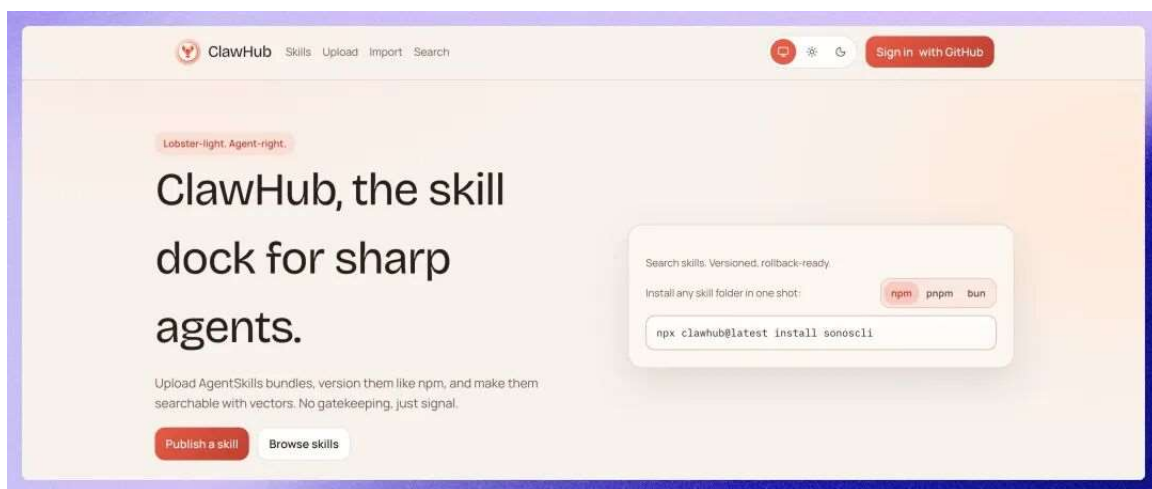
OpenClaw 出厂只有基础对话能力。通过安装 Skills，你可以让它学会浏览网页、定时执行任务、对接外部 API、控制智能家居等等。

内置 Skills（安装 OpenClaw 后自带的）：

- 浏览器控制：让 OpenClaw 能打开网页、搜索信息、提取网页内容
- 定时任务：设置"每天早上 8 点帮我查新闻"之类的自动化任务
- Webhooks：接收来自外部系统的事件通知，触发相应操作

社区 Skills：

OpenClaw 有一个社区 Skills 市场，有点像 App Store 但是给 AI 用的。全球开发者在上面分享自己制作的 Skills。



数据说话，截至目前，已经有 1.6 万+个 Expert Agent（专家代理）被创建。社区的活跃度相当高。

安装一个社区 Skill 的命令类似：

```
openclaw-cn skills install <skill-name>
```

比如想安装一个能帮你管理日程的 Skill:

```
openclaw-cn skills install @community/calendar-manager
```

你配置好一个 Skill，比如"每天下午 5 点帮我汇总今日邮件并写一份摘要"，它就会每天准时执行。你不需要每次都手动告诉它。

一次配置，永久复用。这就是自动化的魅力。

OpenClaw 有三样核心本事——**记得住你**（持久化记忆）、**看得懂图片和文件**（多模态交互）、**学得会新技能**（Skills 扩展）。Skills 是第三样，也是 OpenClaw 从"能用"到"玩出花"的关键。

七、使用技巧篇——3 个让 OpenClaw 更好用的设置

OpenClaw 装好了，进阶功能也了解了。

现在给你 3 个实用技巧，让你的 OpenClaw 用起来更顺手、更高效。

技巧 1：开启持久化记忆——让它越用越懂你

默认情况下，OpenClaw 每次新开对话都是"失忆"的，它不记得你叫什么名字、做什么工作、有什么偏好。每次都得重新介绍自己。

开启持久化记忆后，它会记住你。

你告诉它"我是做产品经理的，平时写 PRD 比较多"，下次你说"帮我写个需求文档"，它会直接用 PRD 的格式来写，不需要你再解释。

你告诉它"我喜欢简洁的回复风格，不要啰嗦"，以后它的回复就会变得精炼。

配置方式：在 WebUI 的设置页面中找到"Memory"或"记忆"相关的选项，开启即可。

或者直接在对话中告诉它："请记住我是 XXX，我的偏好是 XXX。"开启持久化记忆后，它就真的记住了。

技巧 2：常用命令备忘——建议存到手机备忘录

把下面这几个命令存起来。日常使用中你会经常用到。

命令	功能	使用频率
<code>openclaw-cn gateway --port 18789</code>	启动 OpenClaw 网关	每次需要手动启动时
<code>openclaw-cn gateway status</code>	查看网关运行状态	不确定是否正在跑时

命令	功能	使用频率
<code>openclaw-cn dashboard</code>	打开浏览器控制台	日常使用入口
<code>openclaw-cn configure</code>	重新配置（换模型/换 Key 等）	需要修改设置时
<code>openclaw-cn doctor</code>	诊断工具——自动检查配置和运行状态	出问题时第一个跑
<code>openclaw-cn --version</code>	查看当前版本号	确认版本时
<code>openclaw-cn plugins list</code>	查看已安装的插件列表	管理插件时
<code>openclaw-cn plugins install <name></code>	安装新插件	扩展功能时

其中 `openclaw-cn doctor` 特别有用。OpenClaw 出了任何问题，先跑一下 `doctor`。它会自动检查你的配置文件、网络连接、模型状态、依赖版本等，然后告诉你哪里有问题、怎么修。

就像去医院之前先做个体检，省时省力。

技巧 3：保持更新——一行命令，永远最新

OpenClaw 更新非常频繁。

给你几个时间点感受一下：

- 2026.2.17 版本：深度集成 Sonnet 4.6 模型，支持 1M 超长上下文
- 2026.2.21 版本：新增 Gemini 3.1 支持
- 2026.2.23 版本：新增 Claude Opus 4.6 支持，安全性大幅提升

基本每周都有新版本。新功能、Bug 修复、安全更新、新模型支持，全在里面。

更新方法：重新执行一次官方安装脚本就行，它会自动覆盖升级到最新版。

Mac/Linux：

```
curl -fsSL https://clawd.org.cn/install.sh | bash
```

Windows：

```
iwr -useb https://clawd.org.cn/install.ps1 | iex
```

就像你手机上点一下"系统更新"一样，脚本会自动识别你已安装的版本并升级，不需要手动卸载。

建议每 1-2 周更新一次。保持最新版本才能用到最新的功能、最好的模型支持和最及时的安全补丁。

OpenClaw 能帮你做什么——3 个我亲测的使用场景

最后，给你看 3 个我自己亲测过的使用场景，让你更直观地感受到 OpenClaw 不只是"能聊天"。

场景 1：147 个文件，3 分钟归档

上周我桌面堆了 147 个文件，各种 PDF、截图、文档混在一起。我以前都是手动整理，每次花半小时以上。

这次我对 OpenClaw 说了一句："帮我把桌面上的文件按类型分类归档到 Documents/Archive 下面，PDF 放一个文件夹，图片放一个，文档放一个。"

3 分钟后，它告诉我："已完成。PDF 43 个归入 Archive/PDF/，图片 67 个归入 Archive/Images/，Word/Excel 28 个归入 Archive/Documents/，其余 9 个归入 Archive/Other/。"

我打开文件夹确认了一下，分类准确率 100%。以前半小时的活，3 分钟搞定。

场景 2：英文论文速读，含翻车

我把一篇 24 页的英文论文丢给它，说："用中文给我写一个 500 字的结构化摘要，重点讲方法论和主要结论。"

2 分钟后，摘要出来了。结构清晰，核心观点都抓到了。

但说实话，第一次它漏掉了论文里一个重要的实验对比数据。我追问了一句"论文里 Table 3 的实验对比结果呢？"它补充了，这次完整了。

所以我的经验是：AI 生成的摘要拿来作初筛非常好，但关键数据你最好自己再核实一遍。

场景 3：定时新闻推送

配合定时任务 Skills，我设置了："每天早上 8 点，帮我汇总今日 AI 圈的重要新闻，写成 5 条摘要，发到我的飞书上。"

从此以后，每天早上我打开飞书，就有一份热乎的 AI 新闻摘要等着我。用了两周了，漏发过一次（那天通义千问的 API 短暂抽风），其余都准时到达。

你的专属 AI 新闻编辑，准时上岗。

看到了吗？

文件整理、论文速读、定时推送，这些事情，以前要么得你自己动手，要么得找人帮忙。

现在，一句话的事。

我统计了一下，OpenClaw 的 GitHub Issue 里，排名前 10 的问题全是安装相关的，占了所有 Issue 的七成。也就是说，**你今天花 30 分钟搞定安装这件事，就已经跨过了七成人迈不过去的那道坎。**

这哪是工具？这是你的专属数字管家。你睡觉它还在线，你度假它还在岗。

而你，已经拥有了它。

八、写在最后

后续资源

走到这一步的你，已经是一个合格的 OpenClaw 用户了。

我把你接下来可能需要的资源整理在这里：

官方文档：

- OpenClaw 中文社区入门指南：<https://clawd.org.cn/start/getting-started.html>
- OpenClaw 中文社区配置向导：<https://clawd.org.cn/start/wizard.html>
- OpenClaw GitHub 仓库：<https://github.com/openclaw/openclaw>
- 中文版 GitHub 文档：<https://github.com/jiulingyun/clawdbot-chinese>

推荐教程：

- 飞书保姆级对接指南：<https://www.cnblogs.com/catchadmin/p/19592309>
- Windows 部署全攻略补充：<https://www.cnblogs.com/fxio/p/19556075>
- 云部署 vs 本地部署对比：<https://new.qq.com/rain/a/20260205A01RCI00>

懒人方案（不想自己装的看这里）：

MiniMax 上线了 MaxClaw，把 OpenClaw 搬进了网页端，开箱即用，不需要安装任何东西。阿里云和腾讯云也推出了一键部署方案。适合“我不想折腾，只想用”的朋友，但是还是需要自己配置 Skills。且需要订阅套餐才允许使用